OCR Sudoku Solver

Rapport de première soutenance

Groupe 9 : DEGUEST Théodore, BONNEFONT Arthur, BONEFONT Thibaut, BOISARD Emile

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Table des matières

Répartition des tâches **3**

Avancement du projet **4**

Aspects technique **5**
- Chargement et suppression des couleurs d’une image 5
- Rotation manuelle d’une image 6
- Détection de la grille et de la position des cases 8
- Découpage d’une image 12
- Algorithme de résolution d’un sudoku 13
- Réseau de neurones 14

Conclusion **16**

Répartition des tâches

Cela a été difficile pour nous de mettre en place une répartition égale pour chaque élément du groupe que ce soit en termes de difficulté mais aussi de temps . En effet, nous avons eu du mal à évaluer le temps et la difficulté de chaque étape. Finalement, nous avons convenu de la répartition ci-dessous avec un fonctionnement de responsable et suppléant pour les tâches plus complexes.

Tâches	Théodore	Arthur	Thibault	Emile
Chargement d'une image et suppression de couleurs	Responsable		Suppléant	
Rotation manuelle de l'image				Responsable
Détection de la grille et de la position des cases	Responsable			Suppléant
Découpage de l'image		Suppléant		Responsable
algorithme de résolution d'un sudoku			Responsable	
Réseau de neurones		Responsable		

Avancement du projet

Chargement d'une image et suppression des couleurs :

- Le programme peut charger des images de divers type (jpg, png, bmp…)
- La suppression des couleurs est opérationnelle.

Rotation manuelle de l'image :

- La partie sur la rotation manuelle de l'image est terminée entièrement.

Détection de la grille et de la position des cases :

- Les grilles pré-orientées (sans inclinaison) et sans trop de motif autour sont détectées. Chaque case est récupérée et sa position dans la grille est connue.
- Les grilles avec beaucoup de bruit peuvent parfois engendrer des erreurs d'interprétation du programme.

Découpage de l'image :

- Le découpage de l'image est terminé. Nous avons réussi à découper les différentes cases sans avoir des bouts de lignes sur les différentes images.

Algorithme de résolution d'un sudoku :

- L'algorithme de résolution de sudoku est entièrement terminé.

Réseau de neurones

- Le réseau de neurones possède pour l'instant 5 neurones et est capable d'effectuer un XOR.

Aspects techniques

I/ Chargement d'une image et suppression des couleurs (Théodore)

Après avoir fait tout cela, les résultats étaient bons mais toujours pas parfaits. C'est pourquoi nous avons décidé d'améliorer encore notre pré-traitement en ajoutant différentes fonctions, qui pour le moment ne donnent pas les résultats attendus, et qui donc ne sont pas implémentées dans le projet.

La première mais également la plus basique est une gestion du contraste de l'image. Cette fonction prend en paramètre une surface sdl ainsi que le facteur de changement de contraste qui est un nombre décimal compris entre 0 et 2. Un facteur de 1 ne modifie pas l'image, un facteur supérieur à 1 augmente le contraste et un facteur inférieur à 1 diminue le contraste.

```
SDL_Surface* AdjustContrast(SDL_Surface* surface, float contrastFactor) {
    for (int y = 0; y < surface->h; y++) {
        for (int x = 0; x < surface->w; x++) {
           Uint8* pixel = (Uint8*)surface->pixels + y * surface->pitch + x *
                  surface->format->BytesPerPixel;

            // Apply the modifications
            for (int c = 0; c < surface->format->BytesPerPixel; c++) {
                pixel[c] = (Uint8)(contrastFactor * (pixel[c] - 128) + 128);
            }
        }
    }
    return surface;
}
```

Le deuxième ajout est une égalisation d'histogramme, à laquelle nous avons ajouté la possibilité de modifier le nombre de nuances de gris afin de limiter les artéfacts que peut créer ce traitement. Malheureusement, cet algorithme à montré des résultats inégaux en fonction des images prises en paramètre, avec parfois une correction de tous les défauts, et parfois la création de nouveaux défauts. C'est pourquoi nous avons décidé de ne pas l'utiliser pour le moment.

Le dernier ajout que nous avons fait est la normalisation de la luminosité de l'image. Cette fonction est sûrement la plus compliquée des trois à implémenter malgré que son principe soit assez simple. En effet, c'est un algorithme qui prend en paramètre une surface sdl ainsi qu'une luminosité et un écart type à atteindre. Le concept est donc de réduire la luminosité des zones plus claires que la luminosité cible et augmenter celle des zones plus sombres, en conservant toutes les valeurs de luminosité dans la zones permise par l'écart type

```
SDL_Surface* NormalizeImage(SDL_Surface* image, double targetLuminance, double targetStdDev);
```

Ces trois fonctions sont pour le moment à l'état de prototype mais nous comptons les utiliser dans la version finale du projet pour encore améliorer nos résultats.

II/ Rotation manuelle de l'image (Emile)

La rotation de l'image est une étape importante du projet. En effet, comme le prouve l'image 5 de l'annexe du cahier des charges, les grilles de sudoku ne sont pas forcément droites. Par conséquent, il est primordial de les recadrer pour simplifier les étapes suivantes, notamment la reconnaissance des chiffres par le réseau de neurones. Lors de la rotation d'une image il est important de garder toutes les données de l'image principale, il ne faut pas perdre des pixels, que les pixels changent de couleur ou bien même que l'image finale soit rognée.

Pour cela, nous avons créé une fonction "RotateSurface" qui prend en paramètres une surface SDL et un double, qui correspondent respectivement à l'image et à l'angle.

Tout d’abord, la fonction calcule les dimensions de l’image, ce qui va nous éviter d’avoir une image rognée après rotation grâce à ces lignes:

```
    double angle_radians = angle * M_PI / 180.0;
    int dest_width = (int)(original_width * fabs(cos(angle_radians)) + original_height *
fabs(sin(angle_radians)));
    int dest_height = (int)(original_height * fabs(cos(angle_radians)) + original_width *
fabs(sin(angle_radians)));
```

Ensuite elle crée une nouvelle surface(“rotated_surface”) à partir de la fonction “SDL_CreateRGBSurfaceWithFormat. Cette surface a les dimensions calculées précédemment et utilise le même format de pixels que la surface originale.

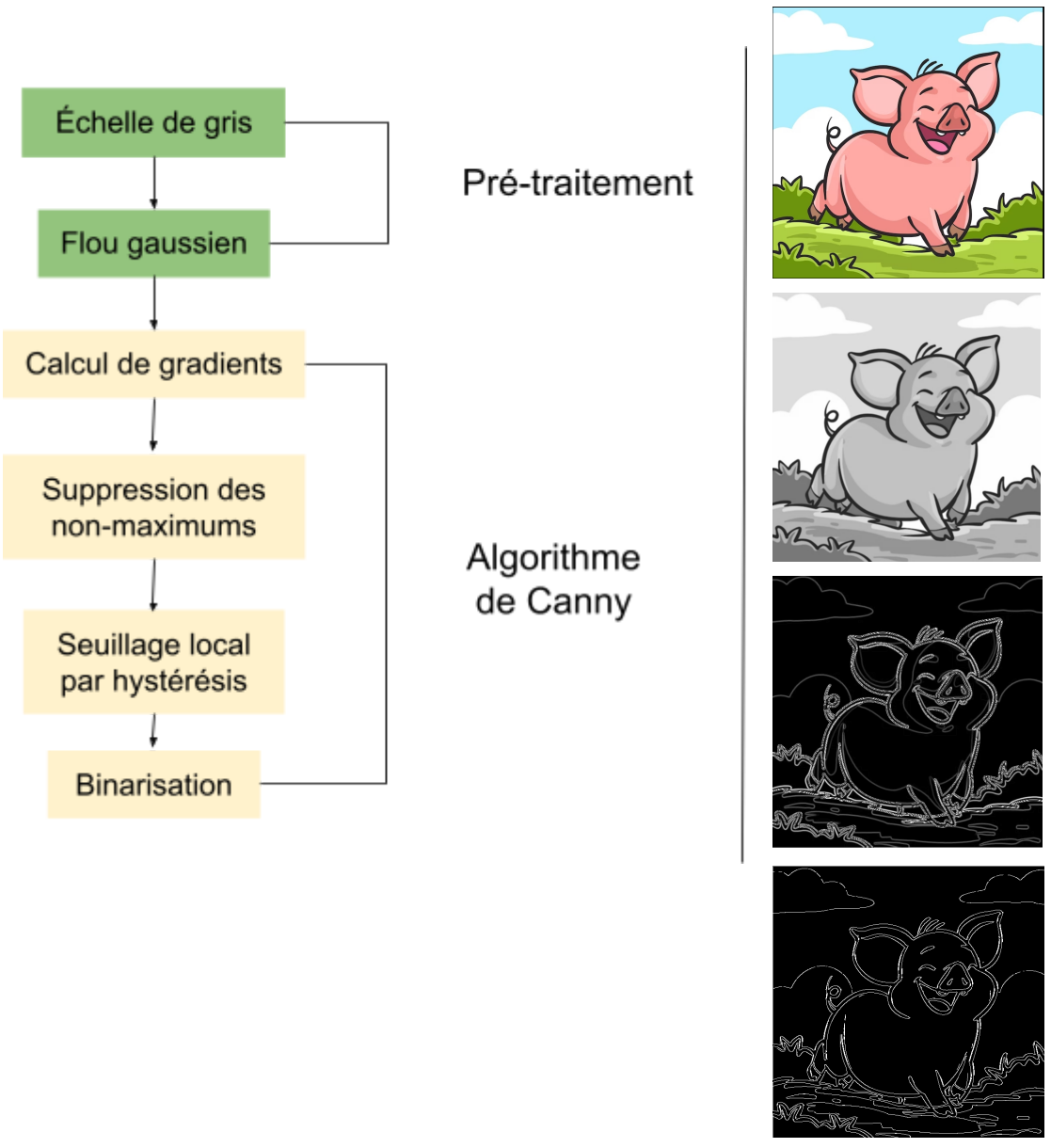
L’étape suivante est celle de l’obtention des couleurs pour cela on utilise cette fois-ci GetPixelRGBA. Grâce à ça, nous pouvons remplir la nouvelle surface avec ces couleurs. Puis, la fonction calcule les coordonnées de destination afin d’avoir une image centrée dans la nouvelle surface. Des rectangles source (src_rec) et destination (dst_rec) sont créés pour définir la région à copier depuis la surface originale et la région où la coller dans la nouvelle surface.

Après tout cela, nous pouvons enfin utiliser SDL_BlitSurfacepour effectuer la rotation. Enfin, La fonction renvoie la nouvelle surface rotative. Si la création de la nouvelle surface ou la rotation échoue, la fonction renvoie NULL.

III/ Détection de la grille et de la position des cases (Théodore)

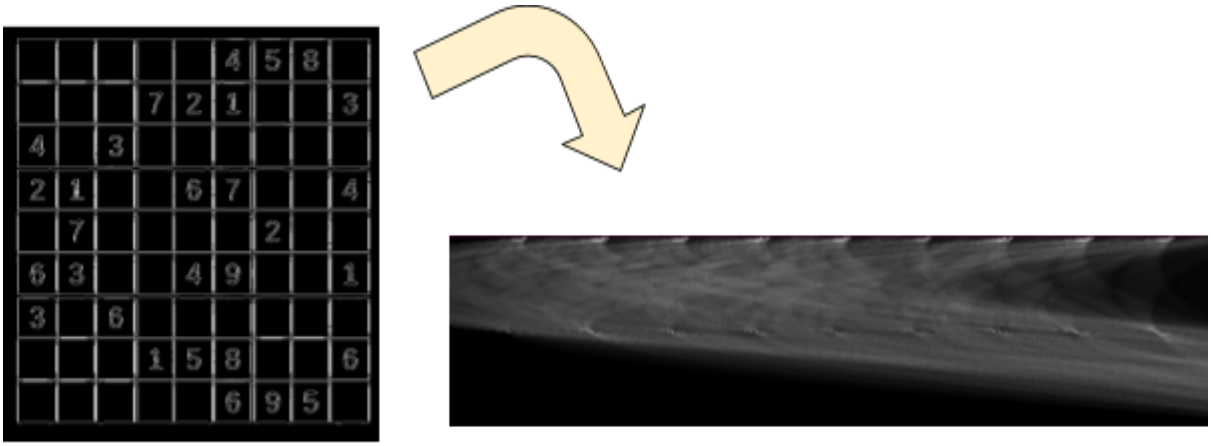
Afin de réaliser la détection de grille, notre algorithme est divisé en deux principales étapes. On utilise tout d'abord la méthode de Canny pour détecter les différents contours présents dans l'image, puis grâce à la transformée de Hough, on détecte les différentes lignes présentes dans ces contours. Finalement nous exploitons ces données pour détecter la position des cases.

Pré-traitement : Afin d'optimiser les résultats obtenus par l'algorithme de canny, nous commençons par appliquer différents filtres.



Après avoir appliqué toutes ces étapes, nous obtenons une image binarisée. Tous les pixel qui font parti d'un contours sont blanc, les autres noirs.

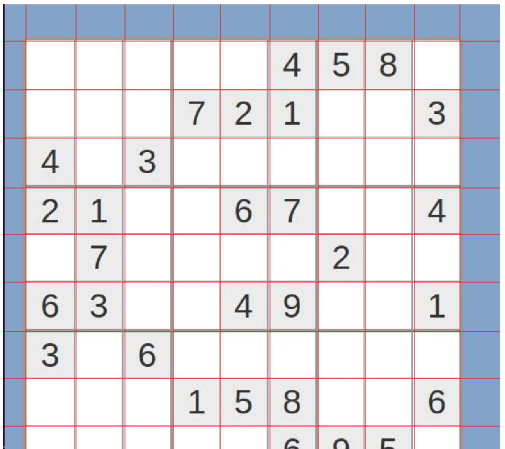
Détection des lignes: Une fois que nous avons trouvé les principaux contours de notre image, nous utilisons un accumulateur et la transformée de hough afin de regarder pour chaque combinaison paramètres (rho et theta) combien de points de contour sont sur la ligne résultante. Ainsi, nous obtenons une matrice qui représente les paramètres des droites dans le plan polaire.



```
for (int j = 0; j < n_y; j ++){
    for (int i = 0; i < n_x; i ++){
        // Check if pixel is white
        if (GetPixelLuminance(image_from_canny, i, j) > 0)
            for (int i_theta = 0; i_theta < n_theta; i_theta ++){
                double theta = i_theta * d_theta;
                double rho = i*cos(theta)+(n_y-j)*sin(theta);
                int i_rho = (int)(rho/d_rho);
                if (i_rho>0 && i_rho<n_rho)
                    accum[i_theta][i_rho] += 1;
            }
    }
}
```

(Avec ce code, nous passons d'un repère cartésien à une représentation polaire)

Une fois ceci fait, nous appliquons un seuillage afin de ne garder que les pixels les plus marqués de l'accumulateur, donc les lignes comportant le plus de points de contour.

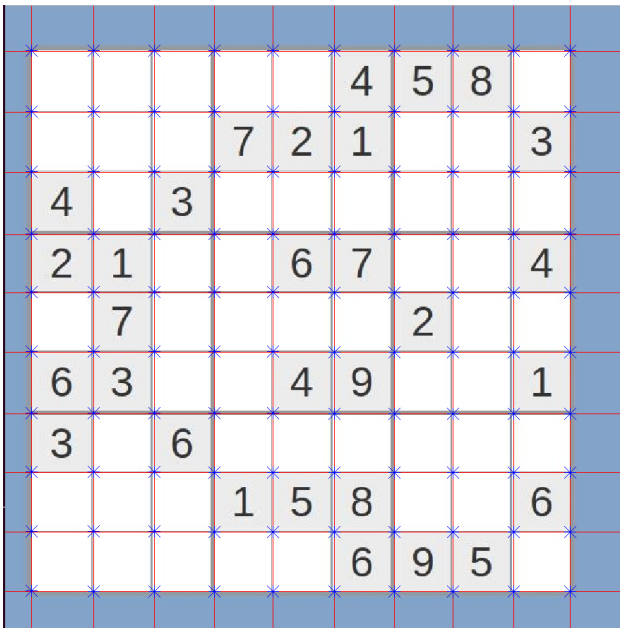


Enfin nous parcourons de nouveau l'accumulateur et lorsque plusieurs pixel blanc se touchent, nous n'en gardons qu'un seul (on fait une moyenne des coordonées du groupe pour savoir lequel garder).
Ainsi, nous n'avons pas de lignes qui sont confondues.
(lignes détectées en rouge ->)

Détection des intersections: À partir des coordonnées polaires que nous avons obtenues, nous calculons pour chaque paire le point d'intersection.

```
        int x = (int)((polar_1.rho * sin(polar_2.theta) - polar_2.rho * sin(polar_1.theta)) /
sin(polar_2.theta - polar_1.theta));
        int y = n_y - (int)((polar_2.rho * cos(polar_1.theta) - polar_1.rho * cos(polar_2.theta)) /
sin(polar_2.theta - polar_1.theta));
```

Ainsi, nous testons pour chaque paire de ligne et nous obtenons tous les points d'intersections.



Pour en déduire les cases, nous partons du point le plus proche de l'origine puis nous regardons s'il y a des points à sa droite et en dessous de lui.
Dans ce cas, c'est le coins haut gauche d'une case. On récupère le point le plus proche à sa droite et en dessous de lui pour déduire les dimensions de la case, puis on passe au point suivant (à sa droite).
Sinon, on passe à la ligne suivante en regardant le point le plus proche de l'origine (donc le point le plus en haut à gauche des points restants).

Nous pouvons alors utiliser la fonction que nous avons créée pour récupérer une partie de l'image. Puisque notre algorithme itère de haut en bas et de gauche à droite, nous pouvons facilement les numéroter.


case_38.png


case_41.png





Enfin, pour améliorer l'interface, nous avons centralisé la gestion du solver, de la détection de grille et du xor dans le fichier main.

```
theodore@theodore-ubuntu:~/SudokuSolver$ ./main help
Usage: ./main <command> [<options>..]

Commands :
    canny: run canny filter
    solver: run sudoku solver
    xor: run xor neural network
    hough: run hough line detection
    grid: run grid detection detection (canny then hough)
    rotate: rotate an image
Options:
    -d: display image information
    -z: resize image (keep proportions), to fit in the screen (CANNY, HOUGH, GRID)
    -t: display processing time
    -o <filename>: store output in a file
    -i <filename>: choose input from a file
    -s: run step by step (CANNY, HOUGH, GRID)
    -h <value>: set high threshold to value (float) (CANNY, HOUGH, GRID)
    -l <value>: set low threshold to value (float) (CANNY, HOUGH, GRID)
    -b <value>: set blur straight to value (int) (CANNY, HOUGH, GRID)
    -x <value>: set nb iteration of xor to value * 10000 (int) (XOR)
    -p <value>: set displaying step of xor to value * 1000 (int) (XOR)
    -r <value>: set hough threshold to value (int) (HOUGH, GRID)
    -a <value>: set rotation angle to value (float) (ROTATE)
```

Ceci nous permet d'avoir la main sur de nombreux paramètres de nos différents sous programmes.

```
theodore@theodore-ubuntu:~/SudokuSolver$ ./main grid
0 : image_07.jpeg
1 : image_01.jpeg
2 : image_05.jpeg
3 : images_08.png
4 : pig.jpg
5 : image_09.jpg
6 : forme.png
Choose a file with index : 1
```

IV/ Découpage de l’image (Emile)

Le découpage de l’image permet de simplifier le travail de reconnaissance des chiffres dans les différentes cases de la grille de sudoku. En effet, ce découpage permettra au réseau de neurones de se concentrer sur une case à la fois et donc un chiffre.

Pour cela, nous avons conçu la fonction `SaveSurfaceSample`,qui prend en paramètres une surface SDL (`source_surface`), les coordonnées `(x, y)` d'un échantillon à extraire de cette surface, ainsi que sa largeur (`width`) et sa hauteur (`height`). De plus, elle prend en entrée le nom du fichier dans lequel l'échantillon extrait sera enregistré au format PNG (file_name).

La fonction commence par vérifier si la surface source (`source_surface`) est valide. Si elle est nulle, la fonction renvoie `EXIT_FAILURE`, indiquant une erreur.

Ensuite, elle crée une nouvelle surface SDL (`sample_surface`) en utilisant les dimensions spécifiées (`width` et `height`). Cette surface sera utilisée pour stocker l'échantillon extrait de la surface source.

Après avoir créé la surface de destination, la fonction copie la zone spécifiée par les coordonnées `(x, y)` avec la largeur et la hauteur données depuis la surface source vers la surface de destination en utilisant la fonction `SDL_BlitSurface`. Comme le montre le code ci-dessous :

```
    SDL_Rect sample_rect = {x, y, width, height};
    if (SDL_BlitSurface(source_surface, &sample_rect, sample_surface, NULL) != 0)
        return EXIT_FAILURE;
```

Enfin, la fonction tente de sauvegarder la surface de destination au format PNG en utilisant la fonction `IMG_SavePNG`. Si cette opération échoue, la fonction libère la mémoire allouée pour la surface de destination et renvoie `EXIT_FAILURE`. Si la sauvegarde est réussie, la mémoire est également libérée, et la fonction renvoie `EXIT_SUCCESS`, indiquant que l'opération s'est déroulée avec succès.

V/ Algorithme de résolution d’un sudoku (Thibaut)

Pour l’algorithme de résolution du sudoku, nous avons décidé d’utiliser le backtracking. En effet, il nous a paru être la meilleure solution, car il est à la fois suffisamment optimisé pour résoudre n'importe quelle grille quasi instantanément mais également parce qu’il est relativement simple à comprendre et à implémenter. Le principe de base de cet algorithme est le suivant :

- déterminer la case à remplir, c’est-à-dire la première case vide sur notre chemin
- remplir cette case avec une valeur comprise entre 1 et 9 telle que cette valeur respecte les règles du sudoku
    - si une valeur qui respecte les règles du sudoku existe, on passe à la case suivante
    - sinon, on revient en arrière et on essaye d’autres valeurs jusqu’à en trouver une valide
- on répète récursivement ces deux étapes précédentes jusqu’à trouver un résultat valide si il en existe un

L’implémentation s’est donc faite de manière assez simple : nous avons la fonction récursive solve_sudoku ainsi que la fonction isValid que l’on appelle à chaque récursion afin de tester la valeur que nous essayons d’ajouter à la grille.

En ce qui concerne l’utilisation de ce programme, il vous suffit simplement d’appeler le fichier compilé en passant une grille de sudoku au format texte en paramètre. Pour ce faire, nous avons dû créer une fonction qui traduit ce fichier texte en une matrice d’int afin de pouvoir l’utiliser dans notre programme. Le résultat quand à lui peut être affiché dans la console comme demandé dans le cahier des charges et donc prendre cette forme ci:

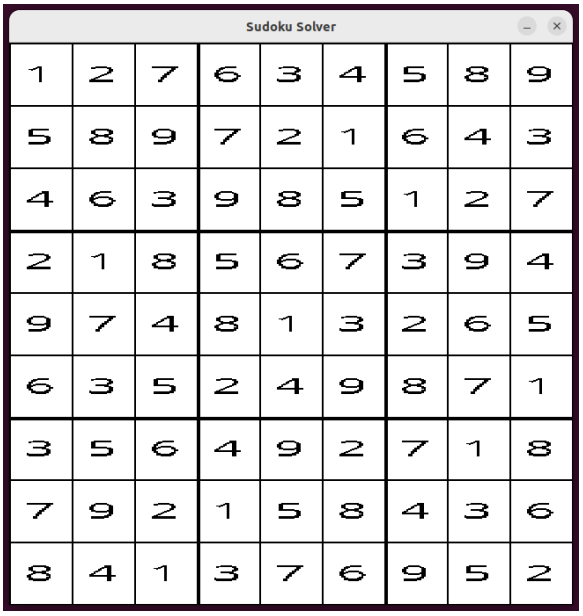
```
thigab@thigab-Lenovo-IdeaPad-S340-14IWL:~/epita/projetS3/SudokuSolver/editImageProcessing$ ./main solver
0 : grid_00
1 : grid_02
2 : grid_empty
3 : grid_01
Choose a file with index : 0
Chosen file is : grids/grid_00

 127 634 589
 589 721 643
 463 985 127

 218 567 394
 974 813 265
 635 249 871

 356 492 718
 792 158 436
 841 376 952
```

Mais nous avons choisi de rajouter également la possibilité d'afficher le résultat directement dans une image:

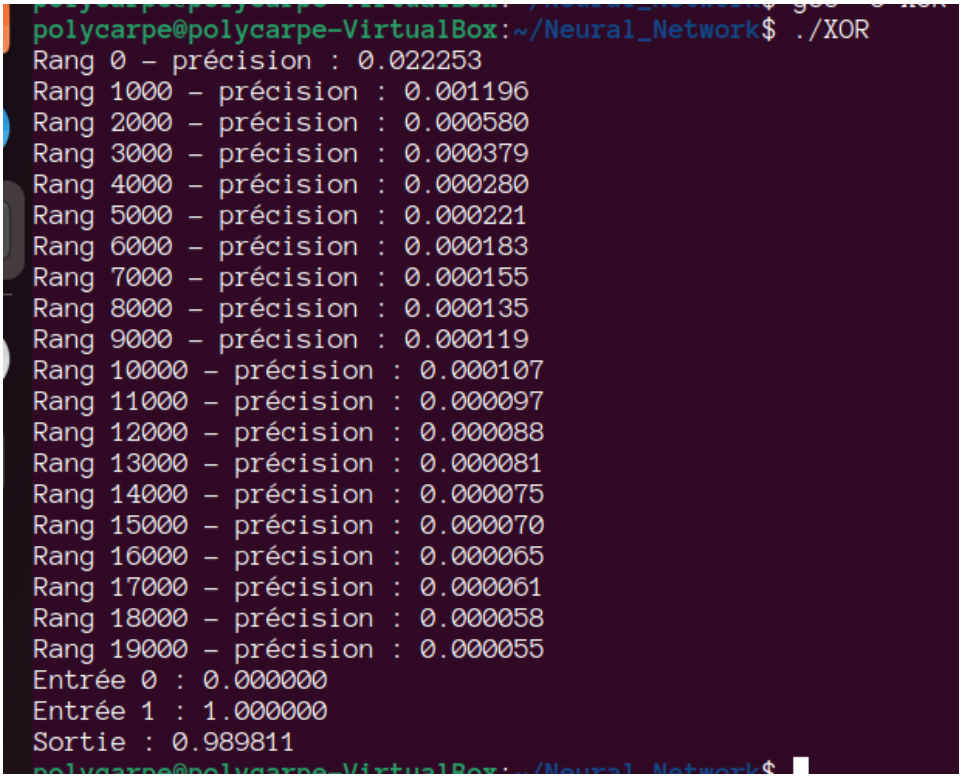


VI/ Réseau de neurones (Arthur)

En ce qui concerne le réseau de neurones, la première partie à été d'essayer de comprendre comment cela pouvait fonctionner, pour cela, nous avons commencé par lire le site qui nous avait été donné, cependant étant un peu indigeste à la première lecture nous nous sommes rabattus sur les millions de vidéos et de sites disponibles sur internet (celui d'IBM par exemple qui est plutôt bien expliqué). Une

fois les bases mieux comprises, nous avons pu retourner au site donné et réaliser un premier test de réseau de neurones : **un réseau à deux neurones**.

Son but étant majoritairement de tester le principe de convergence des neurones, il était très simple et ne pouvait que répondre à l'input (0,1). Pour celui-ci, nous avons décidé d'utiliser la fonction sigmoïde (surtout par souci de simplicité) et parce qu'elle était connue pour marcher relativement bien. Après de nombreuses tentatives, de recherches pour faire compiler la librairie math.h", de segmentation fault et de programmes qui me renvoient des nombres aléatoires, nous y voilà enfin! Nous avions enfin des neurones qui marchaient, le programme renvoyait bien un nombre de plus en plus proche de 1 au fur et à mesure des essais successif.



(ce qui est appelé précision ici est simplement le taux d'erreur)

Le plus dur étant déjà passé, il ne restait plus qu'à adapter le code déjà existant pour qu'il ait 5 neurones et soit capable de résoudre un XOR. Pour cela, on a décidé de garder le sigmoïde et de faire une version qui transportait le résultat en nombre entier, cependant avec cette méthode, on ne pouvait pas observer la convergence, nous avons donc décidé de revenir avec la version en nombre flottant. Le code actuel renvoie donc un chiffre après la virgule.

Afin de simplifier le code, on a décidé de créer une structure « neurone » qui stocke les poids et le biais de chaque neurone. Pas forcément la plus utile pour un XOR avec 5 neurones mais qui deviendra indispensable pour un réseau avec plus de neurones. On a également décidé de laisser le choix à l'utilisateur de choisir le nombre d'essais, ainsi que l'écart d'affichage entre eux. Ce qui nous a posé le plus de problèmes ici à l'ajustement des poids des neurones, puisqu'on a dû l'adapter à la nouvelle structure de neurone.

Conclusion

Malgré les difficultés que nous avons rencontrées, notamment pour ce qui est de la répartition des tâches, nous avons réussi à terminer toutes les étapes à temps. De plus, pour ce qui est de la soutenance finale, nous sommes confiants à l'idée de finir entièrement ce projet.